Lecture 05.14: Abstraction in Python (Abstract Base Classes)

📌 Executive Summary

Enforcing design contracts via abc module, ABC metaclass, @abstractmethod decorator, and building robust interfaces.

🔬 Theoretical Foundations & Principles

1. Abstract Base Classes (ABCs)

An **Abstract Base Class** specifies an architectural interface contract that subclasses MUST implement. It cannot be directly instantiated:

```
from abc import ABC, abstractmethod

class BaseInterface(ABC):
    @abstractmethod
    def compute(self):
        pass
```

2. Compile-Time Enforcement

If a subclass inherits from an ABC and fails to override all `@abstractmethod` definitions, attempting to instantiate the subclass raises `TypeError: Can't instantiate abstract class with abstract method`.

💻 Production Implementation & Code

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name: str):
        self.name = name

    @abstractmethod
    def calculate_salary(self) -> float:
```

```
        pass

class Intern(Employee):
    def calculate_salary(self) -> float:
        return 15000.0  # Fixed stipend

class FullTimeEmployee(Employee):
    def __init__(self, name: str, base_salary: float, bonus: float):
        super().__init__(name)
        self.base = base_salary
        self.bonus = bonus

    def calculate_salary(self) -> float:
        return self.base + self.bonus

intern = Intern("Charlie")
fte = FullTimeEmployee("Dana", 80000.0, 15000.0)
print(f"{intern.name} Salary: ${intern.calculate_salary():,.2f}")
print(f"{fte.name} Salary: ${fte.calculate_salary():,.2f}")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.